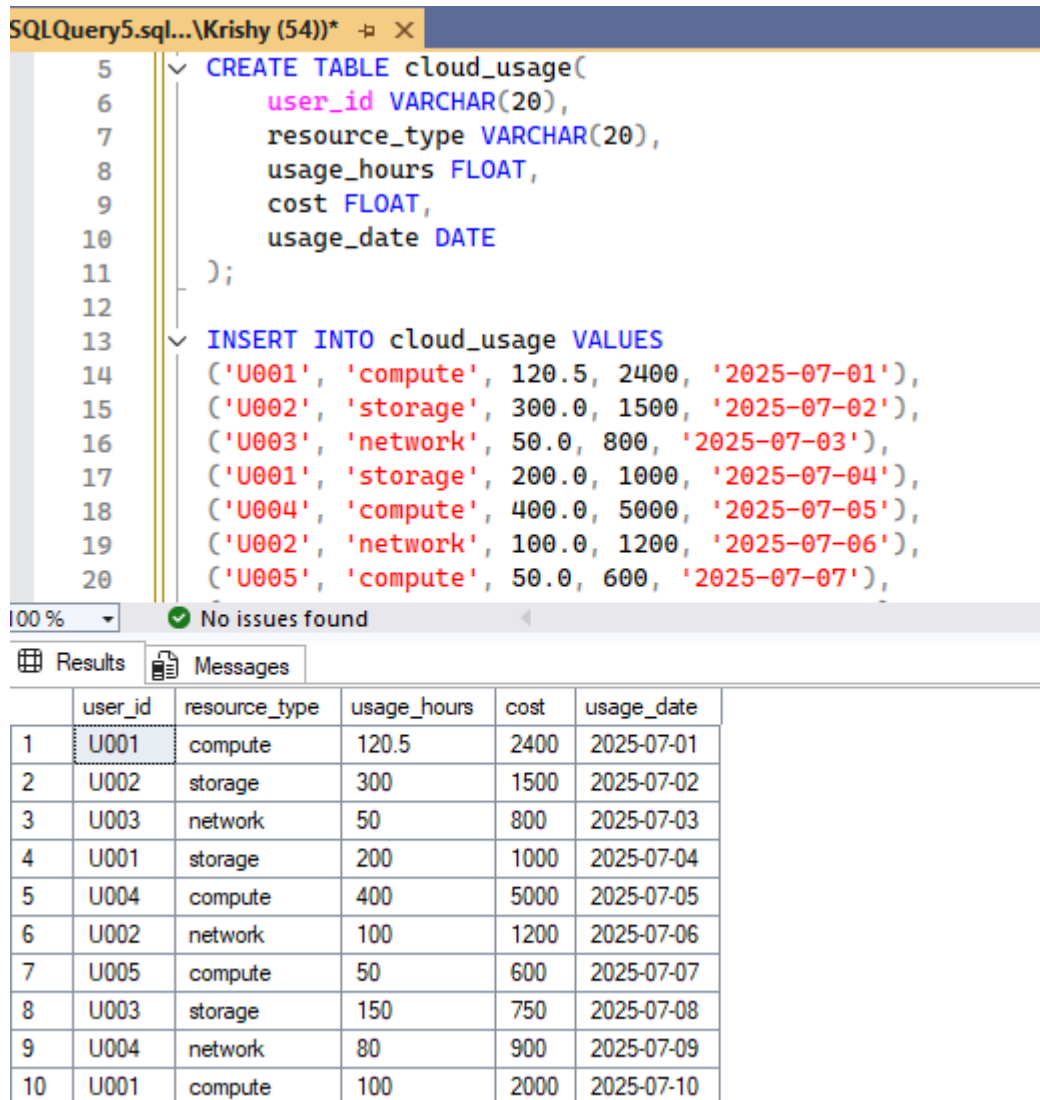


AGGREGATE AND GROUPING FUNCTIONS

1. Create table and insert values

Firstly, Creating the table Cloud usage student as we are finding the cost of each services using aggregate function and grouping along with having functions.



```
SQLQuery5.sql... \Krishy (54))* X
5 CREATE TABLE cloud_usage(
6     user_id VARCHAR(20),
7     resource_type VARCHAR(20),
8     usage_hours FLOAT,
9     cost FLOAT,
10    usage_date DATE
11 );
12
13 INSERT INTO cloud_usage VALUES
14 ('U001', 'compute', 120.5, 2400, '2025-07-01'),
15 ('U002', 'storage', 300.0, 1500, '2025-07-02'),
16 ('U003', 'network', 50.0, 800, '2025-07-03'),
17 ('U001', 'storage', 200.0, 1000, '2025-07-04'),
18 ('U004', 'compute', 400.0, 5000, '2025-07-05'),
19 ('U002', 'network', 100.0, 1200, '2025-07-06'),
20 ('U005', 'compute', 50.0, 600, '2025-07-07'),
```

100 % No issues found

Results Messages

	user_id	resource_type	usage_hours	cost	usage_date
1	U001	compute	120.5	2400	2025-07-01
2	U002	storage	300	1500	2025-07-02
3	U003	network	50	800	2025-07-03
4	U001	storage	200	1000	2025-07-04
5	U004	compute	400	5000	2025-07-05
6	U002	network	100	1200	2025-07-06
7	U005	compute	50	600	2025-07-07
8	U003	storage	150	750	2025-07-08
9	U004	network	80	900	2025-07-09
10	U001	compute	100	2000	2025-07-10

Aggregate functions

Aggregate functions perform a calculation on a set of values (a column) and return a single, summary value. They are typically used with the SELECT statement and are often paired with the GROUP BY and HAVING clause to perform the calculations on subsets of data.

Now we will see one by one...

Sum & Average

- Sum Calculates the total sum of all non-NULL values in a numeric column.
- AVG Calculates the average (mean) value of all non-NULL values in a numeric column.

The screenshot displays three SQL queries in a query editor window. The first query, 'Top 3 Users by Total Cost', uses the SUM function to calculate the total cost for each user, ordered from highest to lowest. The second query, 'Total Cost and average usage of each Resource Type', uses both SUM and AVG functions to calculate the total cost and average usage hours for each resource type, ordered by total cost. The third query, 'Users Spending Over ₹3,000', uses the SUM function to find users whose total cost exceeds 3,000. Below the queries, the 'Results' pane shows the output of the first two queries as tables.

```
1  --Top 3 Users by Total Cost
2  SELECT top 3 user_id, SUM(cost) AS total_spent
3  FROM cloud_usage
4  GROUP BY user_id
5  ORDER BY total_spent DESC
6
7  --Total Cost and average usage of each Resource Type
8  SELECT resource_type, SUM(cost) AS total_spent, AVG(usage_hours) AS avg_usage
9  FROM cloud_usage
10 GROUP BY resource_type
11 ORDER BY total_spent DESC
12
13 --Users Spending Over ₹3,000
14 SELECT user_id, SUM(cost) AS total_spent
15 FROM cloud_usage
16 GROUP BY user_id
```

100 % No issues found

Results Messages

	user_id	total_spent
1	U004	5900
2	U001	5400
3	U002	2700

	resource_type	total_spent	avg_usage
1	compute	10000	167.625
2	storage	3250	216.666666666667
3	network	2900	76.6666666666667

```

13 --Users Spending Over ₹3,000
14 SELECT user_id, SUM(cost) AS total_spent
15 FROM cloud_usage
16 GROUP BY user_id
17 HAVING SUM(cost) > 3000;
18
19
20
21
22

```

100% No issues found

user_id	total_spent
U001	5400
U004	5900

Maximum & Minimum

- Max - Finds the maximum value in a column.
- Min - Finds the minimum value in a column.

SQLQuery7.sql... \Krishy (53))* SQLQuery6.sql... \Krishy (52))* SQLQuery5.sql... \Krishy (54))*

```

1 --Users with High Variability in Cost
2 SELECT
3     user_id, MAX(cost) as Max_cost, MIN(cost) as Min_cost,
4     MAX(cost) - MIN(cost) AS cost_range
5 FROM cloud_usage
6 GROUP BY user_id
7 HAVING MAX(cost) - MIN(cost) > 2000;
8
9 --Users with More Than One Resource Type Used
10 SELECT
11     user_id,
12     COUNT(DISTINCT resource_type) AS resource_variety
13 FROM cloud_usage
14 GROUP BY user_id
15 HAVING COUNT(DISTINCT resource_type) > 1;
16

```

100% No issues found Ln:

	user_id	Max_cost	Min_cost	cost_range
1	U004	5000	900	4100

	user_id	resource_variety
1	U001	2
2	U002	2
3	U003	2
4	U004	2

Count()

- COUNT(*): Counts all rows, including those with NULL values. It's the most common way to count all records in a table.
- COUNT(column_name): Counts only the rows where the specified column_name is not NULL.
- COUNT(DISTINCT column_name): Counts the number of unique, non-NULL values in a column.

The screenshot displays two SQL queries in a query editor window. The first query, titled "--Detecting Idle Users", uses COUNT(*) and SUM to filter users based on their total usage. The second query, titled "--count of the resource used by each user", uses COUNT(*) and COUNT(cost) to aggregate data by user_id. Below the queries, the Results pane shows two tables. The first table, corresponding to the first query, has columns user_id, record_count, and total_usage, with one row for user U005. The second table, corresponding to the second query, has columns user_id, total_entries, and cost_entries, with five rows for users U001 through U005.

```
17  --Detecting Idle Users
18  SELECT
19      user_id,
20      COUNT(*) AS record_count,
21      SUM(usage_hours) AS total_usage
22  FROM cloud_usage
23  GROUP BY user_id
24  HAVING SUM(usage_hours) < 100;
25
26  --count of the resource used by each user
27  SELECT
28      user_id,
29      COUNT(*) AS total_entries,
30      COUNT(cost) AS cost_entries
31  FROM cloud_usage
32  GROUP BY user_id;
```

	user_id	record_count	total_usage
1	U005	1	50

	user_id	total_entries	cost_entries
1	U001	3	3
2	U002	2	2
3	U003	2	2
4	U004	2	2
5	U005	1	1

Thank you!